

Peut-on prédire le succès d'une expédition himalayenne ?

Introduction au Traitement des Données — Projet Kaggle

Romain Ben

Loriziano Soave

Polytech Nice Sophia, MAM3 — Lionel Fillatre, 2025-2026

1 Business goal

Chaque année, des centaines d'expéditions tentent de gravir les sommets de l'Himalaya. Toutes ne réussissent pas. Notre question est la suivante :

*Peut-on prédire si une expédition himalayenne atteindra le sommet, à partir des seules informations connues **avant le départ** ?*

L'enjeu pratique est réel : une agence d'expédition, un assureur ou un chef de cordée voudraient estimer leurs chances en amont, lorsqu'il est encore possible d'ajuster l'équipe, le matériel ou la stratégie. C'est un problème de **classification binaire** (succès / échec), que nous traitons par **régression logistique**.

Le jeu de données. Nous utilisons le jeu de données *Himalayan Expeditions* (Kaggle)^[1], issu de la base historique d'Elizabeth Hawley^[3]. Il recense **11 425 expéditions** décrites par 65 variables. La cible **success** est presque équilibrée : **55 % de succès** contre 45 % d'échec. Cet équilibre est précieux : la régression logistique pourra être entraînée sans rééquilibrage, et l'*accuracy* restera une métrique honnête (un modèle naïf qui prédirait toujours $\hat{y}$ succès $\hat{z}$ n'atteindrait que 55 %).

Une contrainte centrale : éviter le *data leakage*. Notre question impose une discipline. De nombreuses variables du jeu de données décrivent ce qui s'est passé **pendant** ou **après** l'expédition (nombre de membres ayant atteint le sommet, décès, date de sommet, durée totale). Les utiliser reviendrait à $\hat{y}$ prédire le résultat avec le résultat $\hat{z}$. Nous les avons donc systématiquement écartées des prédicteurs : `smtmembers`, `mdeaths`, `smthired`, `hdeaths`, `smtdate`, `termdate`, `totdays`.

Point de départ assumé. Le règlement autorise à s'appuyer sur les contributions publiées. Nous partons donc de la vingtaine de variables retenues par Muhammed Ali Yilmaz dans son notebook Kaggle^[2], plutôt que des 65 colonnes brutes. Notre travail de nettoyage et de sélection s'effectue sur cette base déjà dégrossie, ce qui nous permet de concentrer l'effort sur l'analyse plutôt que sur le tri initial.

2 Team management

Le projet a été mené en binôme par Romain Ben et Loriziano Soave.

Outils. Nous avons travaillé en **Python** (pandas, numpy, matplotlib, seaborn, scikit-learn), organisés en scripts `.py` numérotés correspondant aux étapes du pipeline (exploration, wrangling, visualisation, feature engineering, régression). Le code et les données sont versionnés sur **GitHub**^[5] : https://github.com/BnRomain/Introduction_TraitementDonnees.

Ressources et assistance. Le projet s’appuie sur **Kaggle**, à la fois comme source du jeu de données et comme base de travail (le notebook de M. A. Yilmaz nous a servi de point de départ pour la sélection des variables). Nous avons par ailleurs utilisé une **assistance par intelligence artificielle** sur les parties techniques : aide à l’écriture et au débogage du code Python, et exploration de certaines **corrélations entre variables** pour orienter notre analyse. Chaque résultat suggéré de cette manière a ensuite été **vérifié sur les données** et réinterprété par nos soins : l’IA a accéléré la mise en uvre, mais les décisions (quelles variables garder, comment les justifier) restent les nôtres.

Méthode de collaboration. Nous avons adopté un flux de travail git sur une branche unique, avec synchronisation fréquente (`pull --rebase`) avant chaque contribution pour limiter les conflits. Chaque décision importante (choix d’une variable, rejet d’une feature) est documentée directement en commentaire dans le code, afin que les deux membres partagent le même raisonnement.

Répartition du travail. Nous avons réparti les étapes du pipeline tout en relisant systématiquement le travail de l’autre : Romain a pris en charge l’**exploration** et le **nettoyage** des données (scripts 01 et 02), Lorziano la **visualisation** et le **feature engineering** (scripts 03 et 04). La **régression**, le **rapport** et les **slides** ont été menés en commun, car ce sont les étapes où la discussion à deux apporte le plus.

Planning. Le projet a été mené sur une **semaine de travail intensive**, en suivant l’ordre logique du traitement de données (tableau 1). Ce rythme resserré a été rendu possible par une répartition claire des tâches et par l’usage d’outils nous permettant d’avancer vite sur les parties techniques (voir ci-dessous).

Table 1 – Planning du projet (une semaine).

Jour	Étape
1	Choix du dataset, question métier, mise en place du dépôt Git
2	Exploration et nettoyage des données (<i>wrangling</i>)
3	Visualisation : recherche de relations entre variables
4	<i>Feature engineering</i> et régression logistique (validation)
5	Optimisation, rédaction du rapport et préparation de la soutenance

Démarche. Notre fil conducteur a été de ne jamais retenir une variable ni au feeling ni au feeling z. Chaque choix s’appuie sur une preuve chiffrée tirée des données, et nous avons accepté de **rejeter** nos propres hypothèses lorsque les données les contredisaient (voir section 5). Cette honnêteté est, selon nous, le cœur de la démarche scientifique.

3 Préparation des données (*wrangling*)

Avant toute visualisation ou modélisation, les données brutes doivent être **nettoyées et mises en forme**. Cette étape, souvent invisible dans le résultat final, conditionne pourtant tout le reste : un modèle entraîné sur des données mal préparées apprend des artefacts. Nous avons structuré ce travail en quatre temps et produit deux fichiers intermédiaires (`exped_depart.csv` juste après renommage, `exped_clean.csv` prêt pour le modèle), afin de garder une trace claire de la transformation.

1. Renommage des colonnes. Le fichier brut compte 65 colonnes aux intitulés cryptiques. Conformément au cours (*relabelling columns*), nous avons d’abord réduit le jeu à la vingtaine de

variables retenues par le notebook Kaggle de référence, en leur donnant des noms lisibles (`host` → `host_country`, `success1` → `success`, etc.).

2. Nettoyage des lignes. Nous avons retiré deux catégories de lignes problématiques : les **2 expéditions** dont la saison est `Unknown` (catégorie inexploitable) et les **63 expéditions sans aucun membre** (`totmembers` = 0), qui sont des données douteuses et rendraient impossible le calcul du ratio de Sherpas (division par zéro). Le jeu passe ainsi de **11 425** à **11 362 expéditions**, soit une perte de seulement 0,6 % : le nettoyage est marginal, signe d'un jeu de données déjà de bonne qualité sur les variables qui nous intéressent.

3. Sélection et lutte contre le *data leakage*. Nous ne conservons que les 8 prédicteurs connus **avant le départ** et la cible. Tout le reste est abandonné : d'une part les variables *post-départ* (décès, dates de sommet, durée totale...) qui constitueraient une fuite, et d'autre part les variables sans signal exploitable (`primary_route` et ses 1 272 valeurs uniques, `rope` massivement vide, etc.).

4. Encodage. Le modèle attend des nombres. Nous avons donc converti les booléens (`o2used`, `comrte`, `success`) en 0/1, créé la feature `ratio_hired` (section 5), puis encodé les variables catégorielles `season` et `peakid` en *one-hot*. Pour `peakid` (416 sommets distincts), nous n'avons gardé que les **8 sommets les plus tentés**, regroupant le reste sous `Autre` afin d'éviter une explosion du nombre de colonnes. L'option `drop_first` retire une catégorie de référence (l'AmaDablam pour les sommets, l'automne pour les saisons) : c'est par rapport à elles que s'interpréteront les coefficients du modèle (section 6).

Un choix méthodologique : ne pas standardiser ici. On aurait pu standardiser les variables numériques dès cette étape. Nous ne l'avons **volontairement pas fait** : la moyenne et l'écart-type doivent être calculés sur les seules données d'entraînement, sinon l'information du jeu de test `à fuite` dans les paramètres de prétraitement. La standardisation est donc repoussée à l'intérieur du pipeline de modélisation (section 6), où elle est recalculée à chaque entraînement.

4 Visualisation des données

Avant de modéliser, nous cherchons à **comprendre les relations** entre les variables et le succès. Notre méthode combine plusieurs angles complémentaires, du signal le plus simple à la vue multidimensionnelle.

Outil 1 : le croisement variable / cible. En groupant les expéditions par une variable et en calculant le taux de succès moyen (`groupby().mean()`), on révèle immédiatement un signal. C'est ainsi que l'oxygène est apparu comme le facteur le plus déterminant (figure 1) : le taux de succès **passe de 45 % à 80 %** selon que l'expédition utilise ou non de l'oxygène en bouteille.

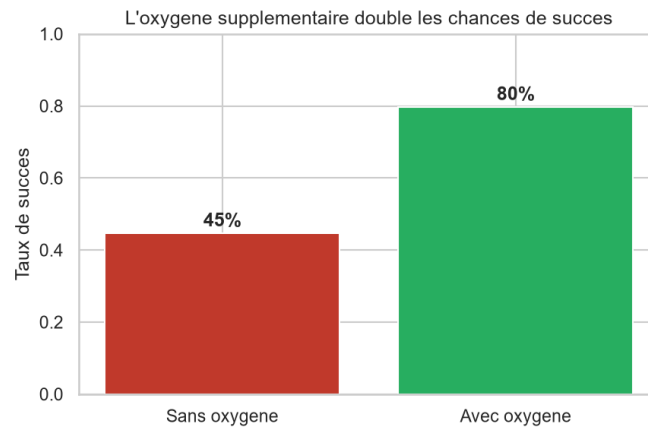


Figure 1 – L’oxygène supplémentaire double presque les chances de succès. C’est le signal le plus fort du jeu de données.

Une vue multidimensionnelle : oxygène × sommet × succès. Le chiffre global n’est qu’une moyenne. Pour croiser **trois dimensions** d’un coup, nous plaçons en abscisse les grands sommets **classés par difficulté croissante** (leur taux de succès sans oxygène, du plus accessible au plus dur) et traçons **deux courbes** : avec et sans oxygène (figure 2). Nous ne gardons que les sommets où chaque groupe compte au moins 30 expéditions, pour des taux fiables. La lecture est frappante : la courbe **avec oxygène** reste haute et presque plate (67 à 88 %), **le sommet importe alors peu** ; la courbe **sans oxygène** s’effondre (51 % au Cho Oyu jusqu’à 6 % à l’Everest), **la difficulté du sommet devient décisive**. L’aire verte entre les deux courbes, c’est-à-dire **la valeur ajoutée de l’oxygène**, se creuse à mesure que le sommet est difficile : c’est une interaction **oxygène × sommet** que la moyenne globale masquait.

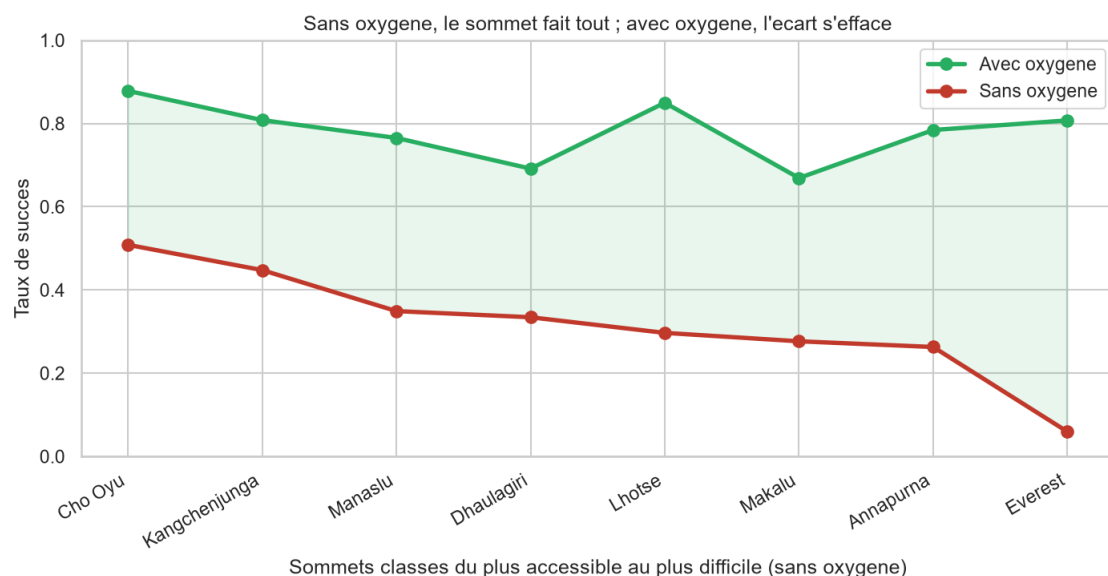


Figure 2 – Taux de succès par sommet, avec et sans oxygène. Sommets classés du plus accessible au plus difficile (taux sans oxygène). La courbe « avec O₂ » reste haute et plate ; la courbe « sans O₂ » s’effondre. L’écart entre les deux (zone verte) mesure l’apport de l’oxygène et grandit avec la difficulté.

Outil 2 : la matrice de corrélation. La heatmap (figure 3) donne la vue d’ensemble des liens *linéaires* entre variables numériques. Elle sert deux objectifs : repérer ce qui corrèle avec la cible (o2used : 0,32 ; camps : 0,28 ; comrte : 0,22), et détecter les **redondances**. On y voit notamment

que `tothired` et `totmembers` sont corrélées (0,58) : les grosses équipes embauchent plus de Sherpas. Cette redondance nous a directement inspiré une feature (section 5).

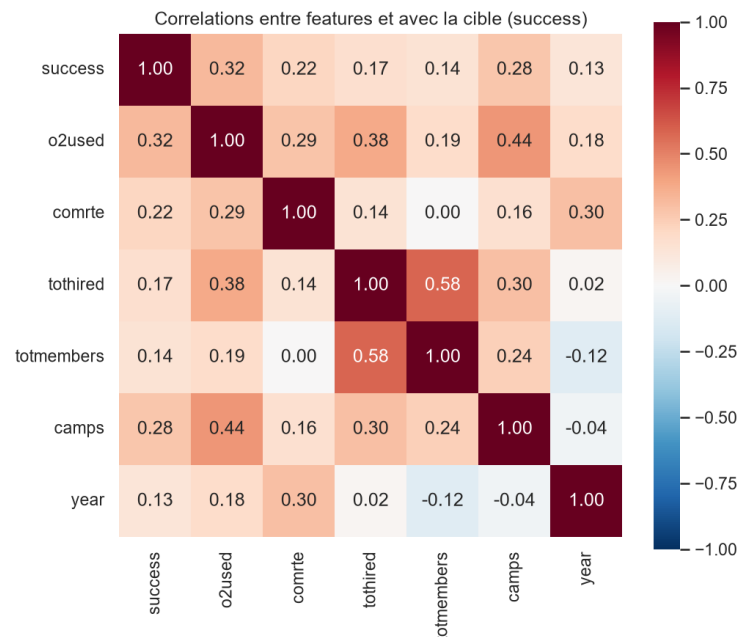


Figure 3 – Corrélations entre prédicteurs et avec la cible. Outil de vue d’ensemble, limité aux variables numériques et aux liens linéaires.

Croiser deux variables : la heatmap 2D. Pour aller plus loin que les corrélations, nous croisons deux variables contre la cible. La figure 4 montre le taux de succès selon la taille de l’équipe (colonnes) et le nombre de Sherpas (lignes). La lecture est éloquent : en parcourant une colonne de bas en haut (équipe de taille fixée), le succès **augmente nettement avec le nombre de Sherpas**, tandis que les lignes restent assez plates. Ce n’est donc pas la taille brute de l’équipe qui compte, mais la *proportion* de personnel d’encadrement.

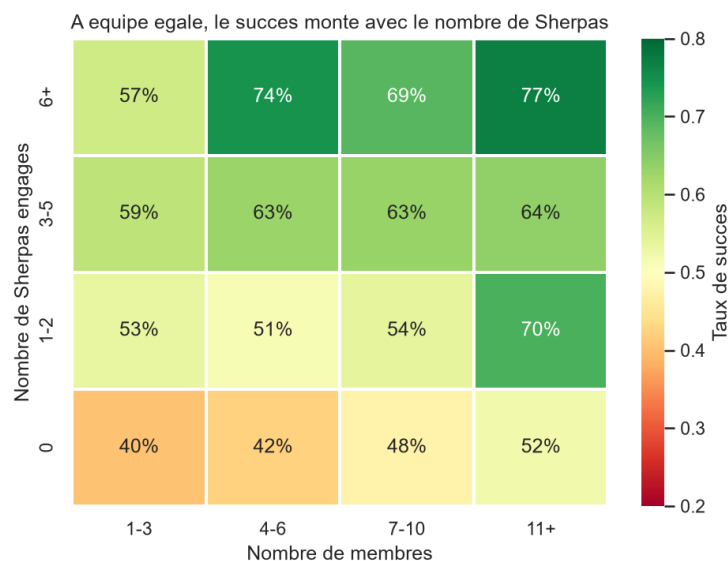


Figure 4 – À taille d’équipe égale, le succès monte avec le nombre de Sherpas. Les cases reposant sur moins de 20 expéditions sont masquées pour ne pas tirer de conclusion sur de trop petits effectifs.

5 Handcrafted features

Cette section illustre notre démarche : pour chaque feature inventée, nous formulons une hypothèse, nous la testons sur les données, puis nous décidons de la garder ou non. Sur trois features bricolées, **une seule a survécu**, et c'est précisément ce qui rend la démarche crédible.

A. `ratio_hired` : la proportion de Sherpas (GARDÉE)

Hypothèse. Suite à la figure 4, ce n'est pas le nombre brut de Sherpas qui compte mais leur proportion. Nous définissons `ratio_hired = tothired / totmembers`.

Test. Le taux de succès croît de façon nette avec le ratio (figure 5) : **49 %** pour un ratio inférieur à 0,5, contre **64 %** pour un ratio entre 0,5 et 1. **Décision : gardée** (avec une réserve révélée plus tard : en présence des autres prédicteurs, son apport devient marginal, voir section 6).

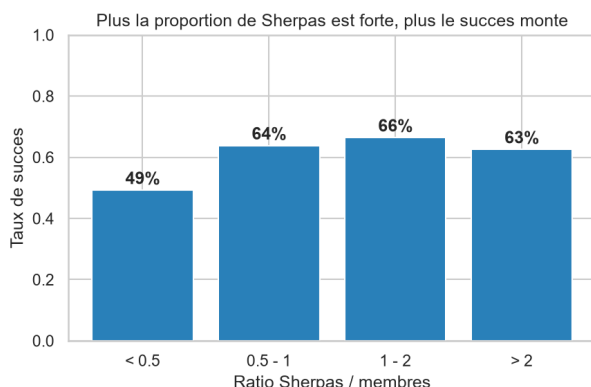


Figure 5 – Plus la proportion de Sherpas par membre est élevée, plus le taux de succès augmente.

B. `o2used` $\times$ `season` : une fausse interaction (REJETÉE)

Hypothèse. La figure 6 semblait montrer que l'effet de l'oxygène *dépend* de la saison : +46 points au printemps, quasi nul en hiver. Cela suggérerait une interaction à encoder.

Test critique. Deux objections nous ont alertés. D'abord les **effectifs** : seulement 40 expéditions n'hiver + O₂ et 9 n'été + O₂, trop peu pour conclure. Ensuite la **physique** : l'oxygène aide partout en altitude, la saison ne change pas la pression partielle d'O₂. Nous avons donc cherché une **variable confondante**, et nous l'avons trouvée : l'oxygène est concentré sur l'Everest (76 % des expéditions Everest), et l'Everest se tente à 88 % au printemps. Résultat : la catégorie n'printemps + O₂ est en réalité composée à 68 % d'expéditions Everest. Ce n'est pas la saison qui module l'effet de l'oxygène, c'est le **sommet tenté**. **Décision : rejetée** (corrélation n'est pas causalité).

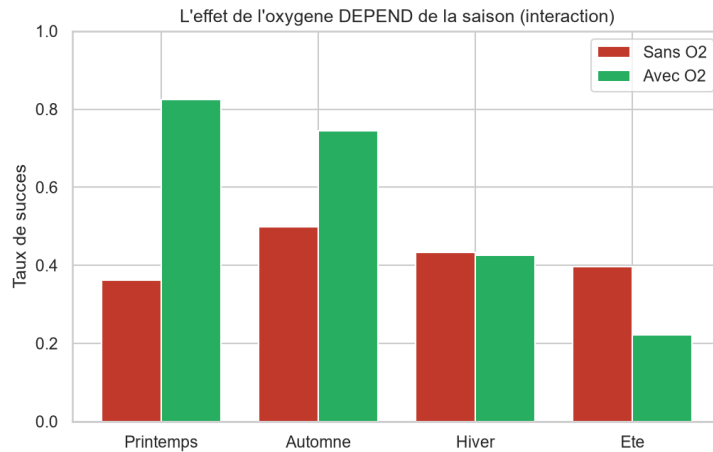


Figure 6 – L'apparente interaction $\text{oxygène} \times \text{saison}$. Le signal est un artefact : la saison est corrélée au sommet tenté, qui porte la vraie explication.

C. `rope_bool` : usage de corde fixe (REJETÉE)

Hypothèse. La variable `rope` (longueur de corde fixe, en mètres) comporte 84 % de zéros. Nous l'avons transformée en booléen $\text{corde utilisée ou non}$, pensant qu'une expédition mieux préparée poserait de la corde.

Test. Les données réfutent l'hypothèse : **55 % de succès sans corde contre 54 % avec** (figure 7), un écart nul, voire inversé. De plus, la corde est posée *pendant* l'ascension (risque de *leakage*) et les 84 % de zéros traduisent sans doute une donnée non renseignée. **Décision : rejetée.**

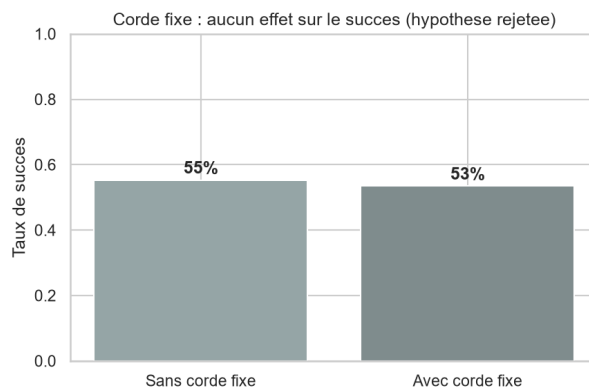


Figure 7 – L'hypothèse $\text{corde fixe} = \text{plus de succès}$ n'est pas vérifiée par les données. Une hypothèse testée et rejetée fait partie d'une analyse honnête.

6 Régression logistique

Quel modèle, et pourquoi. Notre cible `success` est **binaire** (sommet atteint ou non). Ce cadre exclut la régression linéaire (réservée à une cible continue) et rend la **régression logistique** naturelle : elle modélise une probabilité entre 0 et 1. La variante softmax n'aurait de sens que pour plus de deux classes. La régression logistique offre en outre des coefficients **interprétables**, ce qui sert directement notre objectif de comprendre les facteurs de succès.

Variables finales. Après nettoyage (suppression de 2 saisons Unknown et de 63 expéditions sans membre), le jeu de données propre compte **11 362 expéditions**. Les prédicteurs sont : `o2used`, `comrte`, `tothired`, `totmembers`, `year`, `ratio_hired`, ainsi que `season` et `peakid` (regroupé aux

8 sommets les plus tentés + 1 Autre z) encodés en *one-hot*. Nous **excluons délibérément camps** : notre exploration a montré qu'elle contient une fuite (*data leakage*). Sur l'Everest, le taux de succès saute de 22 % (2 camps) à 71 % (3 camps), car l'assaut final part du camp 4 : la variable mesure en réalité 1 jusqu'où l'expédition est montée z, information indisponible avant le départ. La conserver reviendrait à prédire le résultat avec une partie du résultat.

Pipeline anti-fuite. Tout le modèle est encapsulé dans un Pipeline combinant la **standardisation des variables numériques** et la LogisticRegression^[4]. Le point méthodologique clé est que la standardisation est **à l'intérieur** du pipeline : à chaque entraînement, la moyenne et l'écart-type sont recalculés sur les seules données d'entraînement, jamais sur les données d'évaluation. On évite ainsi que l'information de test ne fuite dans le prétraitement (*data leakage*).

Deux façons d'évaluer : Holdout et validation croisée. Nous avons volontairement comparé deux protocoles d'évaluation (tableau 2). Le **Holdout** réserve un seul jeu de test (20 %) : simple, mais son chiffre dépend du tirage. La **validation croisée** (5 plis, **StratifiedKfold**) entraîne et teste le modèle sur 5 découpages différents puis moyenne les scores ; elle utilise toutes les données et fournit un écart-type, donc une estimation plus fiable. Les deux méthodes donnent des résultats **cohérents** (69,7 % contre 69,3 % $\pm$ 1,1 %) : le Holdout tombe dans l'intervalle de la CV, ce qui confirme que la performance ne doit rien à un découpage chanceux.

Table 2 – Comparaison des méthodes d'évaluation (accuracy). La validation croisée et le Holdout concordent ; l'optimisation n'apporte rien de significatif.

Méthode	Accuracy
Baseline (classe majoritaire)	55,0 %
Holdout (split 80/20)	69,7 %
Validation croisée (5 plis)	69,3 % $\pm$ 1,1 %
Modèle optimisé ($C = 3$), test Holdout	69,6 %

Optimisation des hyperparamètres. Nous avons cherché le meilleur paramètre de régularisation C (qui contrôle la complexité du modèle) par **GridSearchCV**, avec une discipline stricte : la recherche se fait par validation croisée **interne au train**, le test Holdout n'étant jamais vu pendant le réglage (sinon on optimiserait sur le juge final). Le meilleur réglage trouvé est $C = 3$, mais son gain est **nul** sur le test (69,6 % contre 69,7 % pour le modèle par défaut). La leçon est instructive : sur une régression logistique déjà bien posée, les paramètres par défaut étaient quasi optimaux, et le meilleur réglage en validation interne n'est pas garanti meilleur sur le test. Optimiser ne crée pas de signal qui n'existe pas ; pour un vrai gain, il faudrait changer de famille de modèle.

Métriques détaillées (validation croisée). Au-delà de l'accuracy, le modèle présente une **précision de 75 %** (parmi les expéditions prédites 1 succès z, 3 sur 4 le sont réellement), un rappel de 66 % et un ROC-AUC de 0,747. La matrice de confusion (figure 8), construite à partir des prédictions *out-of-fold*, montre un modèle équilibré, sans biais marqué vers une classe.

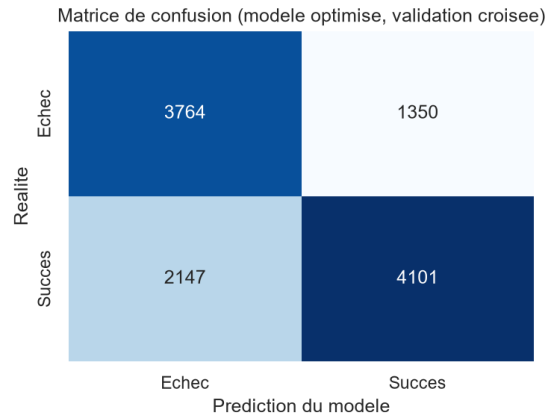


Figure 8 – Matrice de confusion en validation croisée (5 plis). La diagonale concentre les bonnes prédictions.

Interprétation des coefficients. Les variables continues étant standardisées et les autres en 0/1, les coefficients sont comparables entre eux (figure 9). Trois enseignements :

- **L’oxygène domine tout** (`o2used` : +2,03), suivi de la **route commerciale** (`comrte` : +1,39). Ces deux leviers écrasent tous les autres et confirment l’analyse exploratoire.
- Les coefficients `peakid` mesurent la difficulté *par rapport au sommet de référence*, l’AmaDablam. Les plus marqués sont négatifs, ce qui est logique (l’AmaDablam est l’un des plus accessibles) : l’**Everest** (−1,97) et le **Manaslu** (−1,63) ressortent comme les plus difficiles.
- **Notre feature `ratio_hired` s’effondre** (+0,02, négligeable). Pourtant nette en analyse univariée (section 5), elle perd tout pouvoir une fois `o2used` et `comrte` présents : ces derniers absorbent son signal (les expéditions bien encadrées sont aussi celles qui prennent de l’oxygène et passent par une agence). C’est un rappel classique : une variable utile seule peut devenir inutile en présence d’autres. Nous la conservons par transparence, mais son apport réel est marginal.

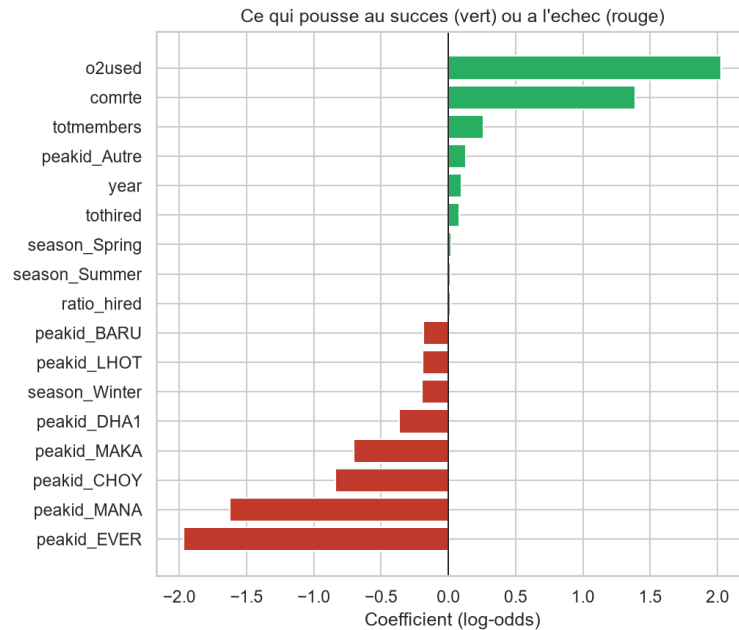


Figure 9 – Coefficients du modèle final. En vert ce qui augmente la probabilité de succès, en rouge ce qui la diminue. L’oxygène et la route commerciale dominent ; les sommets s’interprètent relativement à l’AmaDablam.

7 Conclusion

Notre analyse dégage une hiérarchie claire des facteurs de succès d’une expédition himalayenne. L’**usage de l’oxygène** domine très largement, suivi de la **route commerciale** (encadrement logistique) et de la **difficulté intrinsèque du sommet**. Le modèle, validé par validation croisée, atteint 69,3 % d’accuracy, loin devant la baseline de 55 %. Ces résultats sont cohérents avec le bon sens alpin, ce qui crédibilise l’approche.

Au-delà des chiffres, le projet illustre deux pièges classiques du traitement de données que nous avons su identifier. Le **data leakage** : certaines variables séduisantes (**camps**) contiennent en réalité le résultat. La **confusion entre corrélation et causalité** : notre $\acute{u}$ interaction $\acute{z}$ oxygène $\times$ saison n’était qu’un reflet du sommet tenté. Dans les deux cas, ce sont le raisonnement métier et le test critique, plus que les statistiques brutes, qui ont permis de trancher.

Limites et pistes. La qualité des données reste inégale (zéros ambigus, dates manquantes). Une amélioration naturelle serait d’intégrer l’**altitude du sommet** comme variable continue (plutôt que **peakid** en one-hot), qui résumerait mieux la difficulté. On pourrait aussi comparer la régression logistique à un modèle non linéaire pour vérifier si des interactions plus fines existent, tout en restant vigilant sur le *leakage*.

8 Références

1. Jeu de données : *Himalayan Expeditions*, Kaggle. <https://www.kaggle.com/datasets/siddharth0935/himalayan-expeditions>
2. Notebook d’inspiration : M. A. Yilmaz, *Himalayan Climb Prediction with ML/DL*, Kaggle. <https://www.kaggle.com/code/muhammedaliyilmazz/himalayan-climb-prediction-with-ml-dl>
3. Source historique : The Himalayan Database (E. Hawley).
4. *scikit-learn* : documentation de LogisticRegression.
5. Dépôt du projet : https://github.com/BnRomain/Introduction_TraitementDonnees